



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Новосибирский государственный технический университет»



**НГТУ
НЭТИ** | **Факультет прикладной
математики и информатики**

Кафедра теоретической и прикладной информатики
Лабораторная работа № 2
по дисциплине «Методы оптимизации»

МЕТОДЫ СПУСКА
(0-го, 1-го и 2-го порядка и переменной метрики)

Бригада 4

ПМ-34 РУБЦОВ АРТЁМ

ПМ-34 КУДЮКОВ МАКСИМ

Преподаватель

ЛЕМЕШКО БОРИС ЮРЬЕВИЧ

Новосибирск, 2026

Содержание	
1 Цель работы.....	3
2 Задание.....	3
3 Результаты.....	4
3.1 Квадратичная функция: $f(x, y) = 100(y - x)^2 + (1 - x)^2$, $\epsilon = 10^{-3}$	4
3.1.1 Выводы	6
3.2 Функция Розенброка: $f(x, y) = 100(y - x^2)^2 + (1 - x)^2$	7
3.2.1 Выводы	11
3.3 Целевая функция: $f(x, y) = 21 + x - 12x^2 + y - 21x^2 + 11 + x - 33x^2 + y - 132$	12
3.3.1 Выводы	15
4 Вывод.....	15

1 Цель работы

Ознакомиться с методами поиска минимума функции n переменных в оптимизационных задачах без ограничений [1,5,7,8].

2 Задание

Метод	Порядок
Метод сопряженных градиентов в модификации Данилина-Пшеничного (Полака-Рибьера)	1
Метод Бroyдена	Переменной метрики

Условие задачи:

Найти максимум заданных функций:

На квадратичной функции: $f(x, y) = 100(y - x)^2 + (1 - x)^2$,

На функции Розенброка: $f(x, y) = 100(y - x^2)^2 + (1 - x)^2$,

На целевой функции:

$$f(x, y) = \frac{2}{1 + \left(\frac{x-1}{2}\right)^2 + \left(\frac{y-2}{1}\right)^2} + \frac{1}{1 + \left(\frac{x-3}{3}\right)^2 + \left(\frac{y-1}{3}\right)^2}$$

4 Вариант

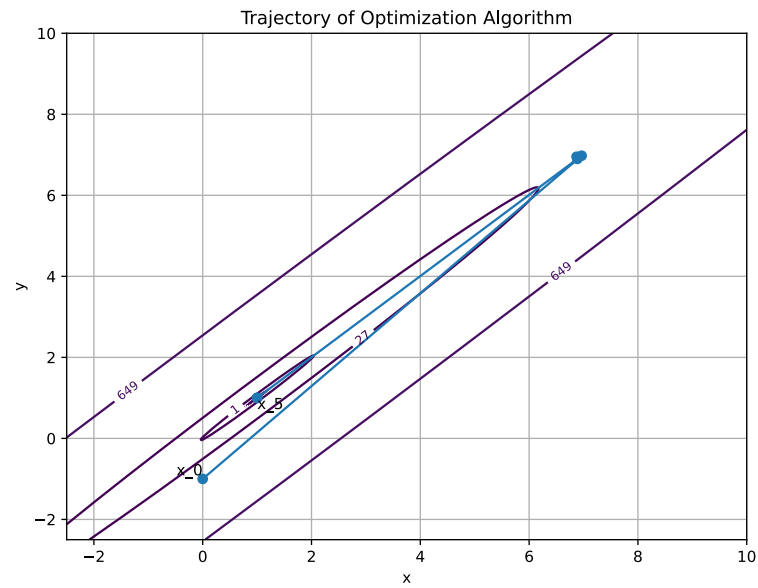
A_1	A_2	a_1	a_2	b_1	b_2	c_1	c_2	d_1	d_2
2	1	1	3	2	3	2	1	1	3

3 Результаты

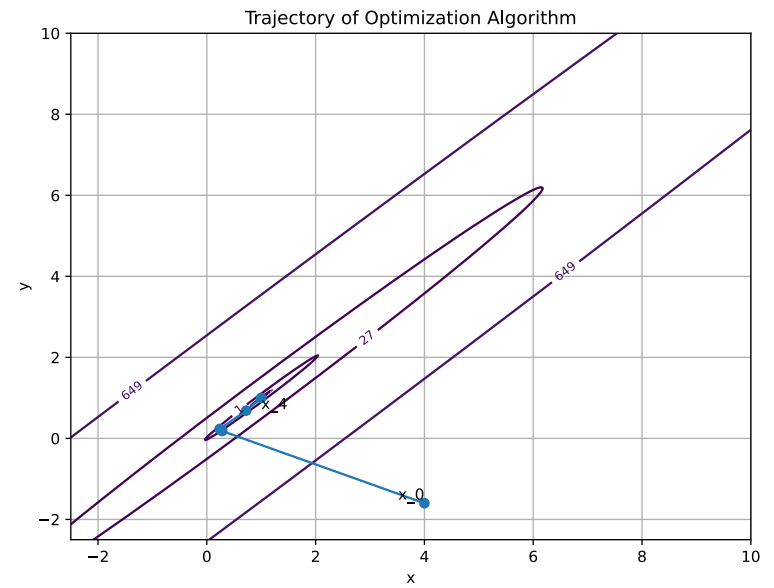
3.1 Квадратичная функция: $f(x, y) = 100(y - x)^2 + (1 - x)^2$, $\text{eps} = 10^{-3}$

Метод сопряженных градиентов в модификации Данилина-Пшеничного											
i	x_0 y_0	x_i	y_i	f_i	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	Угол между (x_i, y_i) и (s_1, s_2)
1	0	-0.64277	-0.651	2.705458	1.638659	1.646872	0.002494	20.44277	20.349	166814.7	179.7786
2	-1	1	1	5.13E-12	4.55E-05	-4.52E-05	1.002506	1.642766	1.650999	2.705458	89.85518
1	4 -1.6	1.211193	1.212252	0.044715	-0.21066	-0.21172	0.002494	115.3888	114.8123	5312567	179.8816
2		1.126927	1.127563	0.016151	-0.20279	-0.20363	0.4	0.084266	0.084689	0.028564	179.8977
3		0.96222	0.96226	0.001428	0.05586	-0.03585	1.3	0.164708	0.165303	0.014723	77.69375
4		0.962384	0.962155	0.00142	0.052897	0.030749	0.002936	0.000164	0.000105	7.28E-06	14.82351
5		0.963951	0.964597	0.001341	0.76871	0.754936	0.053311	0.001568	0.002443	7.90E-05	0.537121
6		1	1	1.15E-17	-6.56E-08	6.68E-08	0.046895	0.036049	0.035403	0.001341	89.48208

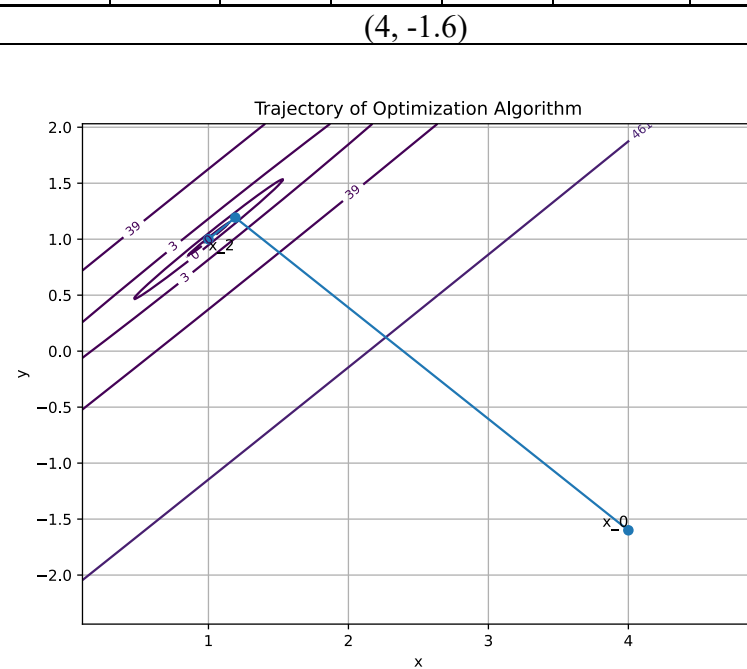
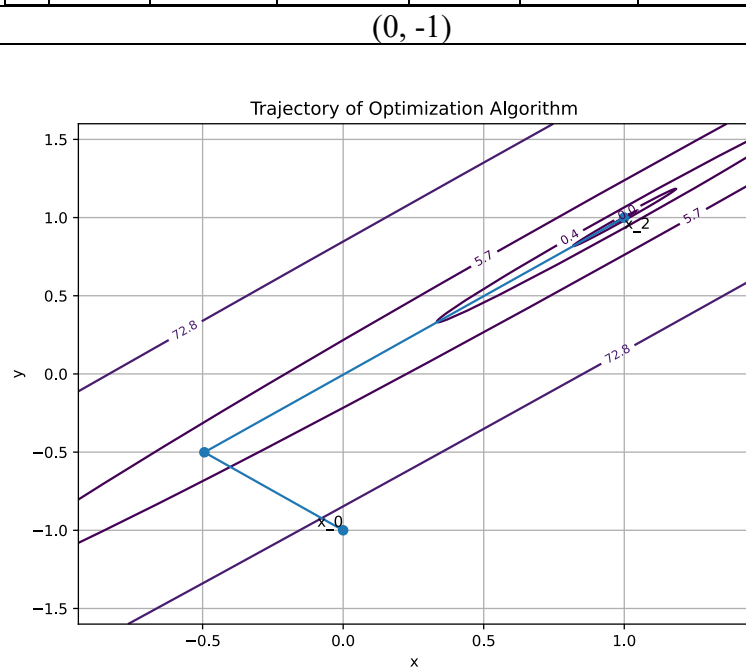
$(0, -1)$



$(4, -1.6)$



Метод Бroyдена														
i	x_0 y_0	x_i	y_i	f_i	λ	$ x_i$ $- x_{i-1} $	$ y_i$ $- y_{i-1} $	$ f_i$ $- f_{i-1} $	eta1	eta2	eta3	eta4	grad1	grad2
1	0	-0.4938	-0.5012	2.2369	0.0025	0.4938	0.4988	98.7631	0.4988	0.4987	0.4987	0.5037	-1.5037	-1.4839
2	-1	1.0000	1.0000	0.0000	1.0025	1.4938	1.5012	2.2369	0.5000	0.5000	0.5000	0.5050	0.0000	0.0000
1	4	1.1920	1.1930	0.0370	0.0025	2.8080	2.7930	3144.96	0.4988	0.4987	0.4987	0.5037	0.1818	0.2022
2	-1.6	1.0000	1.0000	0.0000	1.0025	0.1920	0.1930	0.0370	0.5000	0.5000	0.5000	0.5050	0.0000	0.0000



3.1.1 ВЫВОДЫ

Для квадратичной функции оба метода сходятся к точке минимума (1,1).

Метод сопряжённых градиентов требует больше итераций для достижения заданной точности.

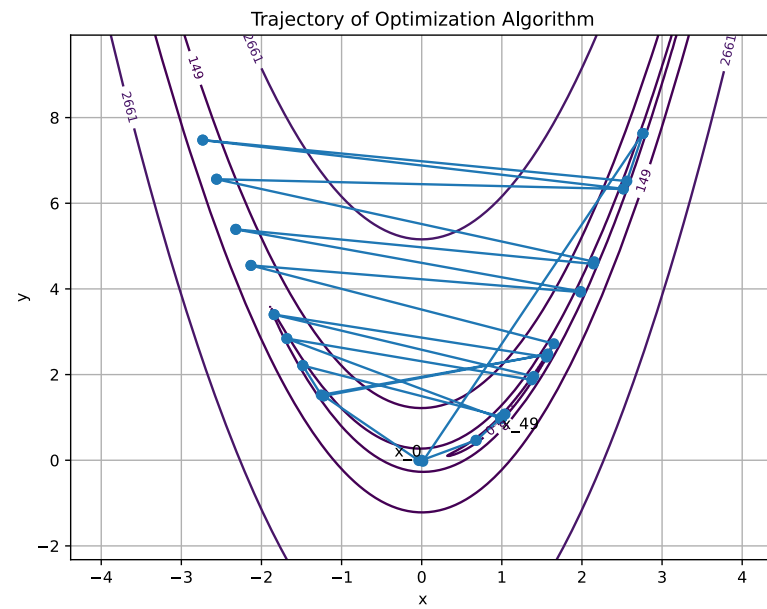
Метод Бroyдена достигает минимума быстрее за счёт использования аппроксимации матрицы Гессе.

Следовательно, для данной функции метод Бройдена показывает более быструю сходимость.

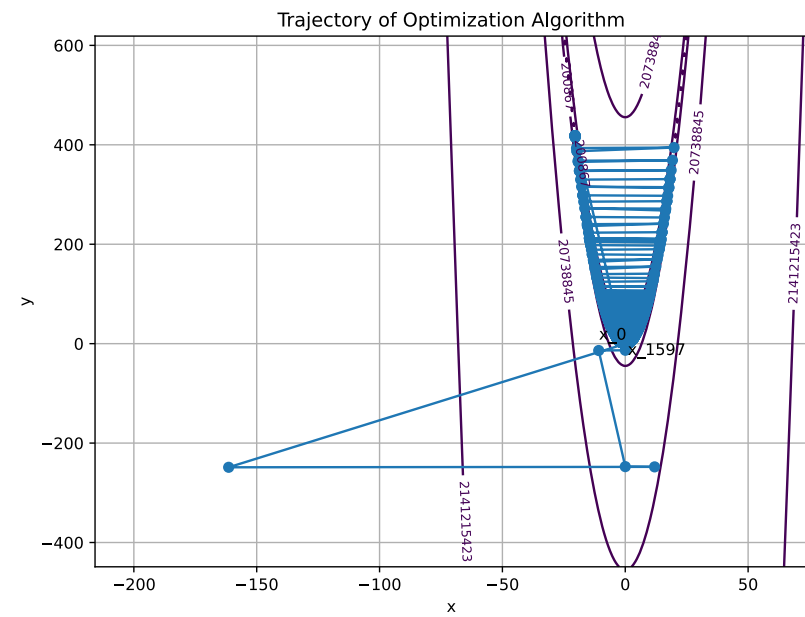
3.2 Функция Розенброка: $f(x, y) = 100(y - x^2)^2 + (1 - x)^2$

Метод сопряженных градиентов в модификации Данилина-Пшеничного											
i	x_0 y_0	x_i	y_i	f_i	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	Угол между (x_i, y_i) и (s_1, s_2)
1	0 0	1.48	2.4	4.623616	1116.548	1377.259	0.3	1.68	2.4	3.023616	7.371048
2		1.592334	2.538564	0.351781	0.810279	-0.53148	0.000101	0.112334	0.138564	4.271835	91.16346
3		1.592723	2.538249	0.35154	-0.12299	-0.39191	0.000519	0.000389	0.000315	0.000241	165.3157
4		1.432833	2.028772	0.246097	-218.253	-643.576	1.3	0.15989	0.509477	0.105444	163.5012
29		0.99678	0.993557	1.04E-05	0.001293	0.002583	0.001004	0.000154	8.02E-05	1.50E-05	18.503
30		0.999998	0.999987	8.78E-09	-0.00103	0.0073	2.489193	0.003218	0.006429	1.04E-05	53.00918
31		0.999994	0.999988	3.31E-11	2.30E-06	4.60E-06	0.000998	3.74E-06	1.87E-06	8.75E-09	18.48522
1	0.5 0.5	2.503753	6.271847	2.262214	0.066718	-0.61392	0.000246	7.103753	0.771847	24552.66	152.0356
2		2.503894	6.270545	2.26181	-1.27619	-6.39132	0.002122	0.000142	0.001302	0.000405	169.5247
3		2.503484	6.2705	2.261405	-0.12676	-0.63491	0.000211	0.000411	4.46E-05	0.000404	169.5269
32		0.999731	0.999323	1.99E-06	0.09334	0.320972	2.402206	0.012422	0.024594	0.000159	28.79707
33		0.999676	0.99935	1.05E-07	0.00013	0.000259	0.000999	5.47E-05	2.76E-05	1.88E-06	18.48385

(0,0)



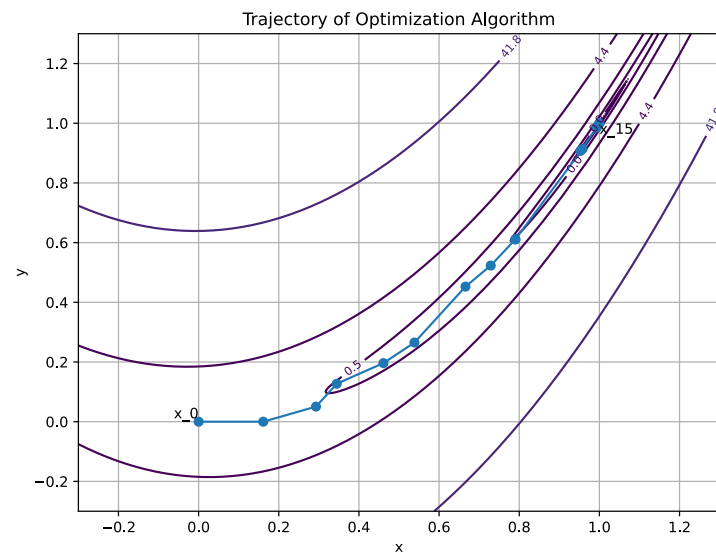
(0.5,0.5)



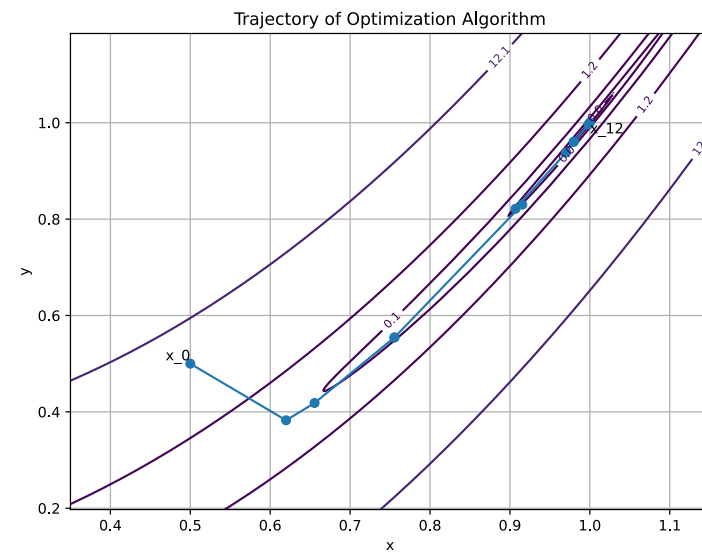
Метод Бройдена $\epsilon_{ps} = 10^{-7}$														
i	x0 y0	x_i	y_i	f_i	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	eta1	eta2	eta3	eta4	grad1	grad2
1	0.0 0.0	1	0.161262	0	0.77111	0.080631	0.161262	0	0.22889	0.889974	0.311221	0.311221	0.119675	-1.09E-06
2		2	0.292836	0.050595	0.62369	0.081285	0.131574	0.050595	0.147419	0.074123	0.03761	0.03761	0.027915	2.703913
3		3	0.350472	0.135729	0.438523	0.9	0.057636	0.085134	0.185167	0.033586	0.026303	0.026303	0.024761	-3.10726
4		4	0.440499	0.179762	0.333425	2.466057	0.090027	0.044033	0.105098	0.195818	0.145704	0.145704	0.112638	1.396625
5		5	0.51178	0.238831	0.291666	0.5	0.071281	0.059068	0.041759	0.087056	0.075811	0.075811	0.067723	3.750029
6		6	0.679178	0.440445	0.14635	7.091319	0.167398	0.201614	0.145315	0.095334	0.103045	0.103045	0.157327	5.019547
7		7	0.674374	0.453999	0.106093	0.097903	0.004805	0.013554	0.040257	0.081886	0.11026	0.11026	0.153456	-0.44061
8		8	0.786303	0.606349	0.059883	2.1	0.111929	0.15235	0.04621	0.345191	0.532715	0.532715	0.831257	3.322833
9		9	0.910057	0.819214	0.016172	1.003172	0.123754	0.212865	0.043712	0.1588	0.273218	0.273218	0.469981	3.092656
10		10	0.903551	0.816661	0.009309	-50.7	0.006506	0.002553	0.006863	0.212104	0.383952	0.383952	0.700019	-0.28551
11		11	0.961304	0.920844	0.002561	1.412711	0.057752	0.104183	0.006748	0.490069	0.936457	0.936457	1.798221	1.176596
12		12	0.993466	0.987689	9.37E-05	0.941104	0.032162	0.066845	0.002467	0.374929	0.735293	0.735293	1.446765	-0.297
13		13	0.999118	0.998165	1.31E-06	0.9	0.005653	0.010476	9.24E-05	0.373734	0.733913	0.733913	1.445169	0.027414
14		14	1.000107	1.000227	2.95E-08	2.1	0.000988	0.002062	1.28E-06	0.517323	1.031914	1.031914	2.063635	-0.00518
15		15	1.000003	1.000006	1.60E-11	1.007447	0.000104	0.000221	2.95E-08	0.561423	1.118326	1.118326	2.23296	0.000103
1	0.5 0.5	0.6198	0.3825	0.1448	0.0023	0.1198	0.1175	6.3552	0.4980	0.4988	0.4988	0.5043	-0.3360	-0.3422
2		0.6555	0.4184	0.1314	0.1055	0.0357	0.0359	0.0134	0.0861	0.0986	0.0986	0.1155	2.2758	-2.2614
3		0.7555	0.5544	0.0864	3.7020	0.0999	0.1360	0.0451	0.0730	0.0580	0.0580	-0.0105	4.4385	-3.2614
4		0.7555	0.5544	0.0864	0.0000	0.0000	0.0000	0.0000	0.1709	0.2547	0.2547	0.3848	4.4385	-3.2614
5		0.9154	0.8300	0.0134	2.2158	0.1599	0.2756	0.0730	0.1801	0.2789	0.2789	0.4485	2.7193	-1.5778
6		0.9070	0.8215	0.0088	0.1688	0.0084	0.0086	0.0046	0.0983	0.1752	0.1752	0.3171	0.2396	-0.2347
7		0.9703	0.9383	0.0019	3.6049	0.0633	0.1169	0.0069	0.3548	0.6686	0.6686	1.2662	1.1611	-0.6289
8		0.9798	0.9606	0.0004	1.1101	0.0095	0.0222	0.0014	-0.1825	-0.3371	-0.3371	-0.6162	-0.2500	0.1070
9		0.9798	0.9606	0.0004	0.0000	0.0000	0.0000	0.0000	0.5527	1.0816	1.0816	2.1211	-0.2500	0.1070
10		0.9983	0.9964	0.0000	0.8234	0.0185	0.0358	0.0004	0.4938	0.9724	0.9724	1.9187	0.1047	-0.0541

11		0.9997	0.9994	0.0000	1.5121	0.0014	0.0031	0.0000	0.5892	1.1721	1.1721	2.3370	-0.0061	0.0028
12		1.0000	1.0000	0.0000	0.7998	0.0003	0.0005	0.0000	0.4985	0.9964	0.9964	1.9967	0.0002	-0.0001

(0,0)



(0.5,0.5)



3.2.1 Выводы

Для функции Розенброка оба метода сходятся к минимуму $(1,1)$.

Метод сопряжённых градиентов требует значительно большего числа итераций.

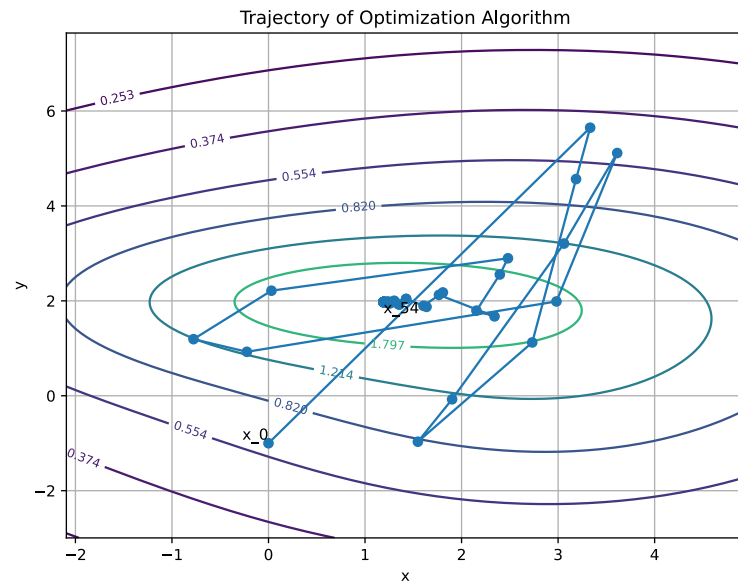
Метод Бройдена обеспечивает более быструю и стабильную сходимость

3.3 Целевая функция: $f(x, y) = \frac{2}{1 + \left(\frac{x-1}{2}\right)^2 + \left(\frac{y-2}{1}\right)^2} + \frac{1}{1 + \left(\frac{x-3}{3}\right)^2 + \left(\frac{y-1}{3}\right)^2}$

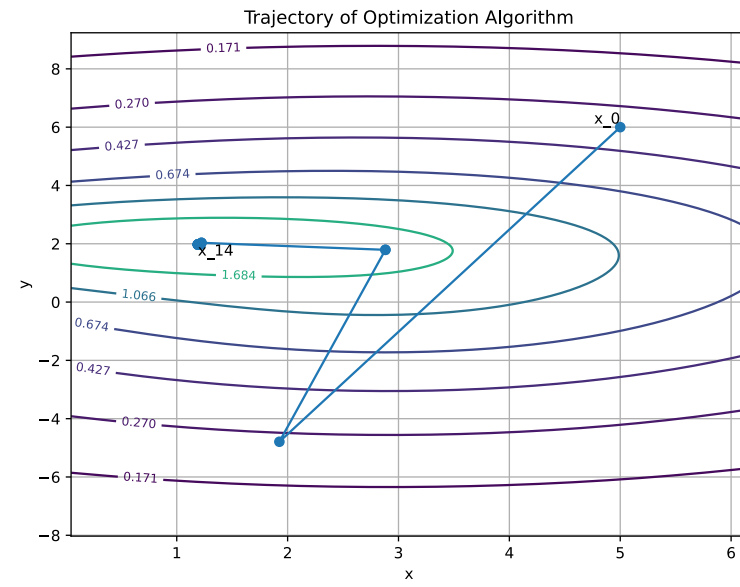
Метод сопряженных градиентов в модификации Данилина-Пшеничного $\text{eps} = 10^{-7}$

i	x0 y0	x_i	y_i	f_i	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	Угол между (x_i, y_i) и (s_1, s_2)	grad1	grad2
1	0 -1	3.13172	5.21241	-	-0.0277	-0.22433	-2.7	0.746279	2.497032	1.23784	156.041	0.017051	0.188695
2		2.940591	3.664558	-	0.26999	-1.99694	6.9	0.19113	1.547853	0.486445	148.9548	0.08327	0.484805
3		2.366026	0.319401	-1.3784	0.04053	0.00427	6.9	0.574565	3.345157	0.395088	166.2979	-0.04306	-0.49099
29		1.190352	1.974413	-	0.00083	0.000219	0.1	4.93E-05	1.55E-05	2.51E-08	106.3973	0.000447	-9.69E-05
30		1.189953	1.974517	-	0.00044	-0.0002	0.47949	0.0004	0.000105	9.45E-08	145.2598	7.64E-05	0.000291
31		1.189945	1.974488	-	-9.65E-05	-0.00029	0.1	7.64E-06	2.91E-05	7.34E-09	167.5014	6.70E-05	0.000175
1	5 6	1.9257	-4.7915	-0.2482	0.0123	0.0844	-12.8000	0.4630	3.2184	0.4548	149.8411	-0.0097	-0.0667
2		2.8819	1.7921	-1.9705	0.1597	4.4960	77.9716	0.9562	6.5836	1.7223	56.0900	0.4831	-0.0702
3		2.8819	1.7921	-1.9705	-0.9662	0.1403	0.0000	0.0000	0.0000	0.0000	139.8608	0.4831	-0.0702
12		1.1899	1.9745	-2.6613	-0.0002	-0.0003	0.5063	0.0008	0.0001	0.0000	166.7283	0.0000	0.0003
13		1.1899	1.9745	-2.6613	-0.0001	-0.0006	0.0000	0.0000	0.0000	0.0000	157.0745	0.0000	0.0003
14		1.1899	1.9744	-2.6613	0.0000	0.0000	0.1234	0.0000	0.0001	0.0000	138.7110	0.0000	0.0000

(0, -1)

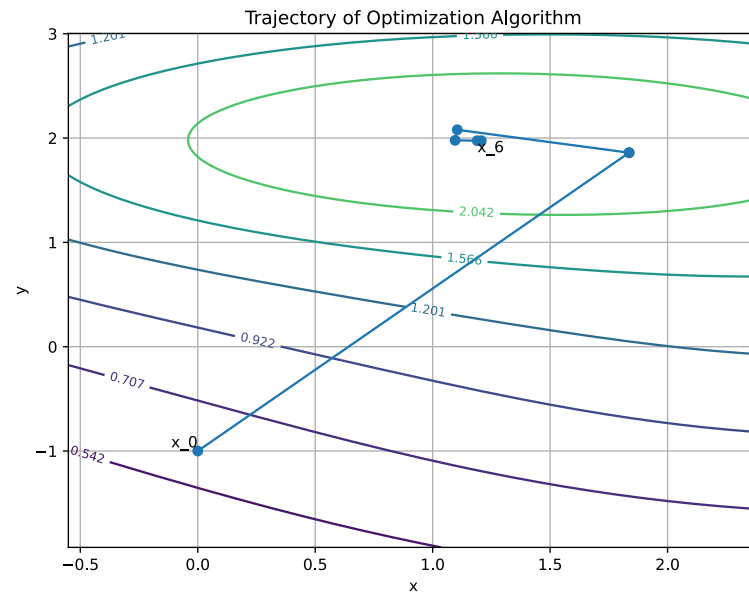


(5,6)

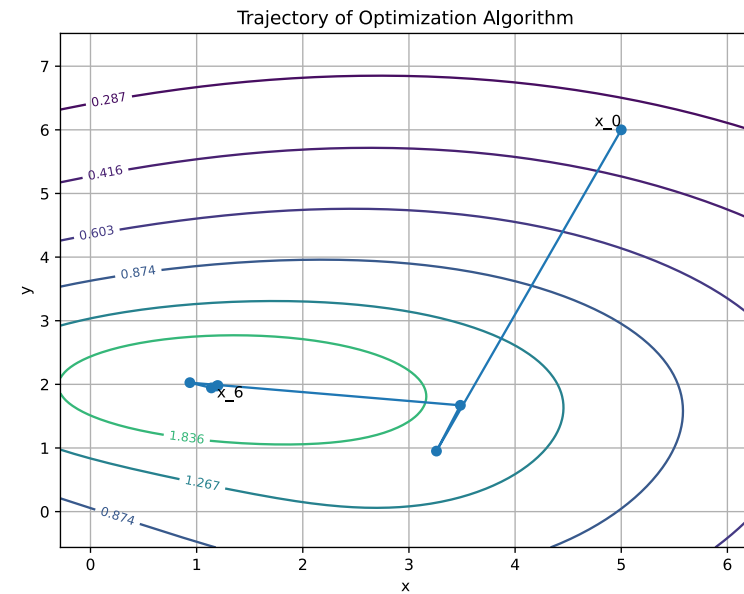


Метод Бройдена														
i	x0 y0	x_i	y_i	f_i	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	Угол между (x_i, y_i) и (s_1, s_2)	grad1	grad2	i
1	0 -1	1.8362	1.8599	2.4857	15.1640	1.8362	2.8599	1.8815	4.6128	8.1710	8.1710	19.4799	0.4159	-
2		1.8362	1.8599	2.4857	0.0000	0.0000	0.0000	0.0000	1.0409	-0.0903	-0.0903	0.3728	0.4159	-
3		1.1049	2.0793	2.6363	1.6000	0.7313	0.2194	0.1506	2.0945	0.4426	0.4426	0.6424	0.0772	0.4142
4		1.0955	1.9781	2.6570	0.4361	0.0094	0.1012	0.0207	1.5296	-0.0281	-0.0281	0.2503	0.0908	0.0085
5		1.2068	1.9744	2.6612	0.8000	0.1113	0.0037	0.0041	1.0413	-0.0178	-0.0178	0.2500	0.0160	0.0011
6		1.1899	1.9744	2.6613	1.0198	0.0169	0.0000	0.0001	1.0622	-0.0198	-0.0198	0.2502	0.0000	0.0001
1	5 6	3.2591	0.9513	1.5848	51.2000	1.7409	5.0487	1.2527	3.2007	5.1281	5.1281	12.9493	0.2550	-
2		3.4844	1.6708	1.6836	0.2000	0.2253	0.7195	0.0988	0.9128	0.1579	0.1579	2.1527	0.4464	-
3		0.9362	2.0269	2.6253	6.4000	2.5482	0.3561	0.9417	3.7916	0.2839	0.2839	2.1583	0.2449	0.1973
4		1.1390	1.9441	2.6581	0.2324	0.2027	0.0828	0.0328	3.0594	1.2108	1.2108	0.9850	0.0511	-
5		1.2005	1.9810	2.6612	0.2000	0.0615	0.0370	0.0031	1.0297	-0.0122	-0.0122	0.2482	0.0105	0.0270
6		1.1899	1.9744	2.6613	1.0055	0.0105	0.0066	0.0001	1.0400	-0.0115	-0.0115	0.2483	0.0001	0.0001

(0, -1)



(5,6)



3.3.1 Выводы

Для заданной целевой функции оба метода находят точку оптимума.

Метод сопряжённых градиентов требует большего числа итераций.

Метод Бройдена достигает решения быстрее.

4 Вывод

Оба метода позволяют решать задачи безусловной оптимизации.

Метод сопряжённых градиентов является устойчивым, но требует больше итераций.

Метод Бройдена обеспечивает более быструю сходимость за счёт аппроксимации матрицы Гессе.

В большинстве рассмотренных случаев метод Бройдена оказался более эффективным по скорости сходимости.

5 Программа

5.1 Main.cpp

```
#include "MSGFR.h"
#include "Broyden.h"
#include <iostream>
#include <iomanip>
int main( )
{
    MSGFR msgfr;
    Broyden broyden;
    msgfr.Init( 2 );
    double *res;
    res = msgfr.Solver( );
    for ( int i = 0; i < 2; i++ )
        std::cout<<std::setprecision(16) << res[i] << std::endl;
}
```

5.2 MatrixOperations.h

```
#pragma once
class MatrixOperations
{
public:
    int n = 2;
    double **AddMatrix( double **mat1, double **mat2 );
    double **MultVecVecT( double *vec1, double *vec2 );
    double **MultMatScal( double **mat, double scal );
    double *MultMatVec( double **mat, double *vec );
};
```

5.3 MatrixOperations.cpp

```
#include "MatrixOperations.h"

double **MatrixOperations::AddMatrix( double **mat1, double **mat2 )
{
    double **res = new double *[n];
    for ( int i = 0; i < n; i++ )
    {
```



```

        res[i] = new double[n];
        for ( int j = 0; j < n; j++ )
            res[i][j] = mat1[i][j] + mat2[i][j];
    }
    return res;
}

double **MatrixOperations::MultVecVecT( double *vec1, double *vec2 )
{
    double **res = new double *[n];
    for ( int i = 0; i < n; i++ )
    {
        res[i] = new double[n];
        for ( int j = 0; j < n; j++ )
            res[i][j] = vec1[i] * vec2[j];
    }
    return res;
}

double **MatrixOperations::MultMatScal( double **mat, double scal )
{
    double **res = new double *[n];
    for ( int i = 0; i < n; i++ )
    {
        res[i] = new double[n];
        for ( int j = 0; j < n; j++ )
            res[i][j] = mat[i][j] * scal;
    }
    return res;
}

double *MatrixOperations::MultMatVec( double **mat, double *vec )
{
    double *res = new double[n] {0};
    for ( int i = 0; i < n; i++ )
        for ( int j = 0; j < n; j++ )
            res[i] += mat[i][j] * vec[j];
    return res;
}

```

5.4 Func.h

```
#pragma once
#include "VecOperations.h"
#include "MatrixOperations.h"
#include <iostream>
class Func
{
    VecOperations opsVec;
    MatrixOperations opsMat;
public:
    int n=2;
    double func(double *x );
    double *gradFunc(double *x );
    double funcInDirection(double *xk, double lambda, double *sk );
    double funcInDirectionEta( double *xk, double lambda, double **etak);
};
```

5.5 Func.cpp

```
#include "Func.h"

double Func::func(double* x)
{
    return 100 * ( x[1] - x[0] ) * ( x[1] - x[0] ) + ( 1 - x[0] ) * ( 1 - x[0] );
    //return 100 * ( x[1] - x[0]* x[0] ) * ( x[1] - x[0]* x[0] ) + ( 1 - x[0] ) * ( 1 - x[0] );
    //return -2 / ( 1 + ( x[0] - 1 ) * ( x[0] - 1 ) / 4.0 + ( x[1] - 2 ) * ( x[1] - 2 ) ) - 1 / ( 1 + ( x[0] - 3 ) * ( x[0] - 3 )
    // / 9.0 + ( x[1] - 1 ) * ( x[1] - 1 ) / 9.0 );
}

double* Func::gradFunc(double* x)
{
    double *res = new double[2] { -200 * ( x[1] - x[0] ) - 2 * ( 1 - x[0] ), 200 * ( x[1] - x[0] ) };
    //double *res = new double[2] { -400 * x[0]*( x[1] - x[0]*x[0] ) - 2 * ( 1 - x[0] ), 200 * ( x[1] - x[0]*x[0] ) };
    //double x1 = x[0], x2 = x[1];

    /////// Знаменатели (те же, что в исходной функции)
    //double g1 = 1.0 + ( x1 - 1.0 ) * ( x1 - 1.0 ) / 4.0 + ( x2 - 2.0 ) * ( x2 - 2.0 );
    //double g2 = 1.0 + ( ( x1 - 3.0 ) * ( x1 - 3.0 ) + ( x2 - 1.0 ) * ( x2 - 1.0 ) ) / 9.0;

    /////// Производные исходной (положительной) функции
    //double dfdx_plus = -( x1 - 1.0 ) / ( g1 * g1 ) - 2.0 * ( x1 - 3.0 ) / ( 9.0 * g2 * g2 );
}
```

```

//double dfdy_plus = -4.0 * ( x2 - 2.0 ) / ( g1 * g1 ) - 2.0 * ( x2 - 1.0 ) / ( 9.0 * g2 * g2 );

//////// Для отрицательной функции градиент меняет знак
//double *res = new double[2];
//res[0] = -dfdx_plus; // минус от производной исходной
//res[1] = -dfdy_plus;

return res;
}

double Func::funcInDirection(double* xk, double lambda, double* sk)
{
    return func( opsVec.AddVec( xk, opsVec.MultVecScal( sk, lambda ) ) );
}

double Func::funcInDirectionEta( double *xk, double lambda, double **etak )
{
    return func( opsVec.AddVec( xk, opsVec.MultVecScal( opsMat.MultMatVec( etak, gradFunc( xk ) ), -lambda ) ) );
}

```

5.6 MSGFR.h

```

#pragma once
#include "Func.h"
#include "VecOperations.h"
#include <iostream>
class MSGFR
{
    int n1 = 2;
    int n;
    double *sk;
    double *sk_1;
    double lambdak;
    double *x0;
    double *xk;
    double *xk_1;
    double eps = 1e-7;
    double eps1 = 1e-14;
    Func f;
    VecOperations ops;
public:
    void Init( int n );
    void FindInterval( double& a, double& b );
    double *Solver( );
    double CalcW( );
}

```

```

double Min( double eps, double an, double bn );
double GoldenRatio( double eps, double an, double bn );
};

```

5.7 MSGFR.cpp

```

#include "MSGFR.h"
#include "iostream"
#include <iomanip>
#include <fstream>
#include <cmath>

void MSGFR::Init( int n1 )
{
    n = n1;
    x0 = new double[2] { 0, -1 };
    sk = ops.MultVecScal(f.gradFunc( x0 ), -1 );
    sk_1 = new double[n];
    xk = new double[n];
    xk_1 = new double[n];
}

void MSGFR::FindInterval(double& a, double& b)
{
    double h = 0.1;
    double lambda0 = 0.5;

    double f0 = f.funcInDirection(x0, lambda0, sk);
    double f1 = f.funcInDirection(x0, lambda0 + h, sk);

    if (f1 > f0)
        h = -h;

    double lambda_prev = lambda0;
    double lambda_curr = lambda0 + h;

    int iter = 0;

    while (iter < 1000)    // защита от бесконечного цикла
    {
        double f_prev = f.funcInDirection(x0, lambda_prev, sk);
        double f_curr = f.funcInDirection(x0, lambda_curr, sk);

        if (f_curr > f_prev)
        {

```

```

        a = std::min(lambda_prev, lambda_curr);
        b = std::max(lambda_prev, lambda_curr);
        return;
    }

    lambda_prev = lambda_curr;
    lambda_curr += h;
    h *= 2;
    iter++;
}

a = 0;
b = 1;
}

double *MSGFR::Solver( )
{
    const double PI = 3.14159265359;
    std::ofstream file1( "out1.txt" );
    file1 << "i\t" << "x\t" << "y\t" << "f\t" << "s1\t" << "s2\t" << "lambda\t" << "|xi - xi-1|\t" << "|yi - yi-1|\t" << "|fi - fi-1|\t" << "angle\t" << "grad1\tgrad2" << std::endl;
    std::ofstream file2( "out2.txt" );
    file2 << 0 << ' ' << x0[0] << ' ' << x0[1] << std::endl;
    double a, b;
    FindInterval(a, b);
    lambdak = GoldenRatio(eps1, a, b);
    xk = ops.AddVec( x0, ops.MultVecScal( sk, lambdak ) );
    xk_1 = x0;
    int k = 0;
    while ( ops.DotProduct( sk, sk ) >= eps )
    {
        if ( k % n1 == 0 )
        {
            sk = ops.MultVecScal( f.gradFunc( xk ), -1 );
        }
        FindInterval(a, b);
        lambdak = GoldenRatio( eps1, a, b );
        for ( int i = 0; i < n; i++ ) xk_1[i] = xk[i];
        xk = ops.AddVec( xk_1, ops.MultVecScal( sk, lambdak ) );
        sk_1 = sk;
        double wk = CalcW( );
        sk = ops.AddVec( ops.MultVecScal( f.gradFunc( xk ), -1 ), ops.MultVecScal( sk_1, wk ) );
        k++;
        std::cout << k << "\t";
    }
}

```

```

file1 << k << "\t";
file2 << k << " ";
for ( int i = 0; i < n; i++ )
{
    std::cout << std::setprecision( 16 ) << xk[i] << ' ';
    file1 << std::setprecision( 16 ) << xk[i] << '\t';
    file2 << std::setprecision( 16 ) << xk[i] << ' ';
}
file1 << std::setprecision( 16 ) << f.func( xk ) << '\t' << sk[0] << '\t' << sk[1] << '\t' << lambdak << '\t';
file1 << abs( xk[0] - xk_1[0] ) << '\t' << abs( xk[1] - xk_1[1] ) << '\t' << abs( f.func( xk ) - f.func( xk_1 ) ) << '\t';
file1 << std::acos( ( xk[0] * sk[0] + xk[1] * sk[1] ) / ( sqrt( ops.DotProduct( xk, xk ) ) * sqrt( ops.DotProduct( sk, sk )
) ) ) * 180 / PI << '\t';
file1 << f.gradFunc( xk )[0] << '\t' << f.gradFunc( xk )[1];
std::cout << std::endl;
file1 << std::endl;
file2 << std::endl;
}
return xk;
}

double MSGFR::CalcW( )
{
    return ops.DotProduct( f.gradFunc( xk ), f.gradFunc( xk ) ) / ops.DotProduct( f.gradFunc( xk_1 ), f.gradFunc( xk_1 ) );
}

double MSGFR::Min( double eps, double an, double bn )
{
    double x, x1, x2;
    double delta = 0.5 * eps1;
    int n = 0;
    while ( bn - an >= eps )
    {
        n++;
        x = 0.5 * ( an + bn );
        x1 = x - 0.5 * delta;
        x2 = x + 0.5 * delta;
        double f1 = f.funcInDirection( xk, x1, sk );
        double f2 = f.funcInDirection( xk, x2, sk );
        if ( f1 < f2 )
        {
            bn = x2;
        }
        else if ( f1 > f2 )

```

```

    {
        an = x1;
    }
    else
    {
        break;
    }
}
return x;
}

double MSGFR::GoldenRatio( double eps, double an, double bn )
{
    double x, x1, x2;
    int n = 1;
    double const C = 0.38196601125010515;
    x1 = an + C * ( bn - an );
    x2 = bn - C * ( bn - an );
    double f1 = f.funcInDirection( xk, x1, sk );
    double f2 = f.funcInDirection( xk, x2, sk );
    while ( bn - an >= eps )
    {
        x = 0.5 * ( an + bn );
        n++;
        if ( f1 < f2 )
        {
            bn = x2;
            x2 = x1;
            f2 = f1;
            x1 = an + C * ( bn - an );
            f1 = f.funcInDirection( xk, x1, sk );
        }
        else if ( f1 > f2 )
        {
            an = x1;
            x1 = x2;
            f1 = f2;
            x2 = bn - C * ( bn - an );
            f2 = f.funcInDirection( xk, x2, sk );
        }
    }
    else
    {

```

```

        break;
    }
}
return x;
}

```

5.8 Broyden.h

```

#pragma once
#include "Func.h"
#include "VecOperations.h"
#include "MatrixOperations.h"
class Broyden
{
    int n = 2;
    double *x0;
    double *xk;
    double *xk_1;
    double **etak;
    double lambdak;
    double eps = 1e-7;
    double eps1 = 1e-7;
    Func f;
    VecOperations opsVec;
    MatrixOperations opsMat;
public:
    void Init( int n1 );
    void FindInterval( double& a, double& b );
    double *Solver( );
    void CalcEtak( );
    double Min( double eps, double an, double bn );
    double GoldenRatio( double eps, double an, double bn );
};

```

5.9 Broyden.cpp

```

#include "Broyden.h"
#include "iostream"
#include <iomanip>
#include <fstream>
#include <cmath>
void Broyden::Init( int n1 )
{

```



```

n = n1;
x0 = new double[2] { 5, 6 };
xk = new double[n] { 0 };
xk_1 = new double[n] { 0 };
etak = new double*[n];
for ( int i = 0; i < n; i++ )
{
    etak[i] = new double[n];
    for ( int j = 0; j < n; j++ )
        etak[i][j] = ( i == j ) ? 1.0 : 0.0;
}

void Broyden::FindInterval( double &a, double &b )
{
    double h = 0.1;
    double lambda0 = 0.5;

    double f0 = f.funcInDirectionEta( xk, lambda0, etak );
    double f1 = f.funcInDirectionEta( xk, lambda0 + h, etak );

    if ( f1 > f0 )
        h = -h;

    double lambda_prev = lambda0;
    double lambda_curr = lambda0 + h;

    int iter = 0;

    while ( iter < 100 )
    {
        double f_prev = f.funcInDirectionEta( xk, lambda_prev, etak );
        double f_curr = f.funcInDirectionEta( xk, lambda_curr, etak );

        if ( f_curr > f_prev )
        {
            a = std::min( lambda_prev, lambda_curr );
            b = std::max( lambda_prev, lambda_curr );
            return;
        }

        lambda_prev = lambda_curr;
        lambda_curr += h;
    }
}

```

```

        h *= 2;
        iter++;
    }

    a = 0;
    b = 1;
}

double *Broyden::Solver( )
{
    double a = 0, b = 0;
    xk = x0;
    int k = 0;
    const double PI = 3.14159265359;
    std::ofstream file1( "out1.txt" );
    file1 << "i\t" << "x\t" << "y\t" << "f\t" << "lambda\t" << "|xi - xi-1|\t" << "|yi - yi-1|\t" << "|fi - fi-1|\t" <<
"eta1\teta2\teta3\teta4\t" << "grad1\tgrad2" << std::endl;
    std::ofstream file2( "out2.txt" );
    file2 << 0 << ' ' << xk[0] << ' ' << xk[1] << std::endl;
    while ( opsVec.DotProduct( f.gradFunc( xk ), f.gradFunc( xk ) ) >= eps )
    {
        for ( int i = 0; i < n; i++ ) xk_1[i] = xk[i];
        FindInterval( a, b );
        lambdak = Min( eps1, a, b );
        xk = opsVec.AddVec( xk_1, opsVec.MultVecScal( opsMat.MultMatVec( etak, f.gradFunc( xk_1 ) ), -lambdak ) );
        CalcEtak( );
        k++;
        std::cout << k << "\t";
        file1 << k << "\t";
        file2 << k << " ";
        for ( int i = 0; i < n; i++ )
        {
            std::cout << std::setprecision( 16 ) << xk[i] << ' ';
            file1 << std::setprecision( 16 ) << xk[i] << '\t';
            file2 << std::setprecision( 16 ) << xk[i] << ' ';
        }
        std::cout << '\t';
        for ( int i = 0; i < n; i++ )
            for ( int j = 0; j < n; j++ )
                std::cout << std::setprecision( 16 ) << etak[i][j] << ' ';
        file1 << std::setprecision( 16 ) << abs( f.func( xk ) ) << '\t' << lambdak << '\t';
        file1 << abs( xk[0] - xk_1[0] ) << '\t' << abs( xk[1] - xk_1[1] ) << '\t' << abs( f.func( xk ) - f.func( xk_1 ) ) << '\t';
    }
}

```

```

        file1 <<etak[0][0] << '\t' <<etak[0][1] << '\t' <<etak[1][0] << '\t' <<etak[1][1] << '\t' << f.gradFunc( xk )[0] << '\t'
<< f.gradFunc( xk )[1];
        std::cout << std::endl;
        file1 << std::endl;
        file2 << std::endl;

    }
    return xk;
}

void Broyden::CalcEtak( )
{
    double *deltaxk = opsVec.AddVec( xk, opsVec.MultVecScal( xk_1, -1 ) );
    double *deltagk = opsVec.AddVec( f.gradFunc( xk ), opsVec.MultVecScal( f.gradFunc( xk_1 ), -1 ) );
    double *buf = opsVec.AddVec( deltaxk, opsMat.MultMatVec( etak, opsVec.MultVecScal( deltagk, -1 ) ) );
    double denominator = 1 / opsVec.DotProduct( buf, deltagk );
    etak = opsMat.AddMatrix( opsMat.MultMatScal( opsMat.MultVecVecT( buf, buf ), denominator ), etak );
    delete[] deltaxk;
    delete[] deltagk;
    delete[] buf;
}

double Broyden::Min( double eps, double an, double bn )
{
    double x, x1, x2;
    double delta = 0.5 * eps;
    int n = 0;
    while ( bn - an >= eps )
    {
        n++;
        x = 0.5 * ( an + bn );
        x1 = x - 0.5 * delta;
        x2 = x + 0.5 * delta;
        double f1 = f.funcInDirectionEta( xk, x1, etak );
        double f2 = f.funcInDirectionEta( xk, x2, etak );
        if ( f1 < f2 )
        {
            bn = x2;
        }
        else if ( f1 > f2 )
        {
            an = x1;
        }
    }
}

```

```

        else
        {
            break;
        }
    }
    return x;
}

double Broyden::GoldenRatio( double eps, double an, double bn )
{
    double x, x1, x2;
    int n = 1;
    double const C = 0.38196601125010515;
    x1 = an + C * ( bn - an );
    x2 = bn - C * ( bn - an );
    double f1 = f.funcInDirectionEta( xk, x1, etak );
    double f2 = f.funcInDirectionEta( xk, x2, etak );
    while ( bn - an >= eps )
    {
        x = 0.5 * ( an + bn );
        n++;
        if ( f1 < f2 )
        {
            bn = x2;
            x2 = x1;
            f2 = f1;
            x1 = an + C * ( bn - an );
            f1 = f.funcInDirectionEta( xk, x1, etak );
        }
        else if ( f1 > f2 )
        {
            an = x1;
            x1 = x2;
            f1 = f2;
            x2 = bn - C * ( bn - an );
            f2 = f.funcInDirectionEta( xk, x2, etak );
        }
        else
        {
            break;
        }
    }
    return x;
}

```

```
}
```

5.10 VecOperations.h

```
#pragma once
#include <iostream>
class VecOperations
{
public:
    int n= 2;
    double *MultVecScal( double *vec, double scal );
    double *AddVec( double *vec1, double *vec2 );
    double DotProduct( double *vec1, double * vec2 );
};
```

5.11 VecOperations.cpp

```
#include "VecOperations.h"
#include <iostream>
double *VecOperations::MultVecScal( double *vec, double scal )
{
    double *res = new double[n];
    for ( int i = 0; i < n; i++ )
        res[i] = vec[i] * scal;
    return res;
}

double *VecOperations::AddVec( double *vec1, double *vec2 )
{
    double *res = new double[n];
    for ( int i = 0; i < n; i++ )
        res[i] = vec1[i] + vec2[i];
    return res;
}

double VecOperations::DotProduct( double *vec1, double *vec2 )
{
    double res = 0;
    for ( int i = 0; i < n; i++ )
        res += vec1[i] * vec2[i];
}
```

```
    return res;  
}
```